

Stage 9 — Construction Profile Registry

Resultado executivo

A Stage 9 implementou exclusivamente o **ConstructionProfile Registry + dispatch declarativo** para os dois módulos Golden: `kitchen-base-2-doors` e `kitchen-golden-upper-800`. O builder profissional deixou de selecionar Carcass e Front Layout por condicionais hardcoded de `moduleDefinitionId`; ele consulta o profile e executa as regras referenciadas pelos resolvers puros existentes.

O resultado preserva a geometria aprovada nas Steps 8–8.2, mantém a separação entre Definition ID e Instance ID, mantém o fallback legado para módulos Kitchen sem profile e bloqueia o caminho profissional quando um profile conhecido está ausente ou produz uma resolução inválida.

A Stage 10 não foi iniciada. Nenhum novo módulo, CAM/CNC, G-code, render, UI, IA, Supabase, Vercel ou Cloudflare foi alterado como parte desta missão.

1. Onde o dispatch estava hardcoded?

O dispatch principal estava em `src/modules/planner-v2/library/families/kitchen/builders.ts`. `buildCarcass()` escolhia entre `GOLDEN_CARCASS_CONSTRUCTION_RULE` e `GOLDEN_UPPER_CARCASS_CONSTRUCTION_RULE` por condicionais de `moduleDefinitionId`; `buildDoors()` fazia seleção equivalente de Front Layout e ainda possuía fallback por `moduleId`. O placement de hardware também recebia diretamente a regra Golden.

A auditoria encontrou, adicionalmente, um gate legado em `hardwareApplicationResolver.ts`. Esse resolver continua puro e mantém sua API histórica para consumidores existentes. A seleção profissional do builder, entretanto, foi externalizada para o Registry; a limitação downstream está explicitamente registrada na auditoria e não altera valores industriais nesta Stage.

2. Qual contrato foi criado?

Foi criado o contrato mínimo `ConstructionProfile`:

```
interface ConstructionProfile {
  id: string;
  moduleDefinitionId: string;
  carcassRule: CarcassConstructionRule;
  frontLayoutRule?: FrontLayoutRule;
  hardwareApplicationRule?: FurnitureAssemblyRule;
}
```

O profile é declarativo. Ele não armazena `ResolvedCarcass`, `ResolvedFrontLayout`, placements, PartDefinitions ou IDs específicos de ocorrências.

3. Onde fica o Registry e como ocorre o bootstrap?

O Registry fica em `library/registry/ConstructionProfileRegistry.ts`. Ele usa um `Map` determinístico indexado por `moduleDefinitionId`, rejeita IDs vazios, duplicatas e rules destinadas a outra definição, expõe lookup/listagem e mantém o conjunto de definições profissionais conhecidas para impedir fallback silencioso.

O bootstrap dos Golden Profiles é único e idempotente no próprio módulo do Registry. Os dois profiles são definidos em `families/kitchen/constructionProfiles.ts` e referenciam os objetos de regra já existentes. Nenhum rule object foi duplicado.

4. Perfis registrados

ModuleDefinition	Profile	Carcass	Front
kitchen-base-2-doors	kitchen-base-2-doors:construction-profile-v1	GOLDEN_CARCASS_CONSTRUCTION_RULE	GOLDEN_2_DOOR_FRONT_LAYOUT
kitchen-golden-upper-800	kitchen-golden-upper-800:construction-profile-v1	GOLDEN_UPPER_CARCASS_CONSTRUCTION_RULE	GOLDEN_UPPER_2_DOOR_FRONT_LAYOUT
kitchen-base-1-door	ausente	—	—

A regra `GOLDEN_71B3550_173H7100_RULE` declara `moduleDefinitionId: kitchen-base-2-doors`, portanto não foi tratada como regra universal de todas as portas Kitchen. Ela é específica do Base e também é disponibilizada como fallback declarativo legado para preservar consumidores antigos que não recebem regra explícita.

5. Como Base e Upper resolvem seus perfis?

A sequência profissional agora é `ModuleDefinition` → `ConstructionProfileRegistry` → `rules` → `pure resolvers` → `builders` → `PartDefinitions`. O builder consulta apenas `options.moduleDefinitionId`. A factory já encaminha `config.id` como Definition ID, enquanto `instanceId` continua sendo usado somente para materializar a ocorrência.

O Base mantém `full-height-above-toe-kick` e `separate-profile-supported-by-feet`. O Upper mantém `full-height`, `toeKickRelation: none`, `toeKickMm = 0` e `toeKick = undefined`. O Front Layout Upper permanece assimétrico conforme o `baseline: reveals 2/4/2/2`, portas `396/396`, centros `-200/+198` e altura `696`.

6. Fallback legado e hard stop profissional

Módulos sem profile, como `kitchen-base-1-door`, continuam usando o comportamento legado. O adaptador `LegacyKitchenDispatch.ts` existe somente para chamadas antigas que não carregam `moduleDefinitionId`; ele não participa do caminho profissional.

Se uma definição profissional conhecida perder seu profile, o builder lança erro e `buildModule()` retorna falha. Se a regra Carcass do profile Upper for mutada para uma configuração inválida, a construção também falha. Em nenhum desses casos `PartDefinitions` aparentemente válidas são fabricadas pelo legacy path.

7. Identidade e persistência

O Registry nunca usa `instanceId` como chave. O teste real pelo store comprovou lookup para `kitchen-base-2-doors` e `kitchen-golden-upper-800`, enquanto `furniture-123456` não encontra profile. Base e Upper na mesma store usam perfis distintos, possuem instance IDs distintos e não compartilham `PartDefinitions`.

A persistência continua armazenando `moduleDefinitionId` no `FurnitureInstance`. O profile não é serializado. No reload, o projeto é reconstruído pelo mesmo caminho `moduleDefinitionId` → `Registry` → `rules` → `build`. O regression check de `updateFurnitureInstance` confirmou que o rebuild continua resolvendo o profile sem expor o ID do profile na instância.

8. Paridade downstream

Os snapshots Stage 9 registram os resultados atuais de Base e Upper e referenciam as evidências aprovadas das Steps anteriores. O Upper continua com **32 PartDefinitions**, sendo **10 peças físicas e 22 componentes de hardware**. O nesting mantém `missingInNesting = []`, `duplicateInNesting = []` e `unknownInNesting = []`. Os acceptance locks anteriores continuam verdes para $A \rightarrow B \rightarrow A$, multi-instância, BOM, cut-list, Joinery, Machining e invariância de movimento/rotação.

A introdução do Registry não altera fórmulas industriais, quantidades de BOM, dimensões de cut-list, relações de Joinery ou coordenadas de Machining. A prova completa está nos logs e snapshots em `evidence/stage9-profile-registry/` e nas suítes Step 8.1/8.2.

9. Mutation checks

Foram adicionados e executados locks para profile vazio, duplicate registration, moduleDefinition mismatch, CarcassRule mismatch, FrontLayoutRule mismatch, remoção temporária do Upper, Carcass inválida, uso de instance ID como chave e source lock contra dispatch hardcoded no builder. As mutações são temporárias e restauradas em memória.

Mutation	Resultado
Duplicate profile	PASS — rejeitado
Wrong CarcassRule	PASS — rejeitado
Remove Upper profile	PASS — build falhou; sem legacy
Instance ID como key	PASS — lookup não encontra profile
INVALID → legacy fallback	PASS — build falhou

10. Open-source provenance

Projeto estudado	Conceito	Decisão Dioris	Código externo copiado?
WoodworkingShop / Cabinet Planner	configuração declarativa separada do engine	adotar profile declarativo antes do builder	Não
Panelizer	dados de painel e validação separados da manipulação visual	manter rules/resolvers fora da UI	Não
dprojects/Woodworking	relações e operações de marcenaria derivadas de configuração	preservar downstream puro e declarativo	Não

Nenhuma dependência ou asset externo foi adicionado.

11. Validação final

A suíte final contém **54 arquivos e 599 testes aprovados**. O TypeScript terminou com exatamente os cinco erros preexistentes em `usePlannerStore.ts`, linhas aproximadas 1089–1093, e nenhum erro novo da Stage 9. O production build terminou com exit code 0.

Item	Resultado	Evidência
ConstructionProfile contract	PASS	src/.../contracts/ConstructionProfile.ts
ConstructionProfileRegistry	PASS	src/.../registry/ConstructionProfileRegistry.ts
Duplicate prevention	PASS	stage9ConstructionProfileRegistry.test.ts
Definition identity	PASS	registry tests e wiring real
Base / Upper registration	PASS	matrix e registry tests
Declarative Carcass dispatch	PASS	source lock e wiring
Declarative Front dispatch	PASS	source lock e wiring
Hardware dispatch	PASS with documented legacy boundary	dispatch audit
Legacy fallback	PASS	registry acceptance
INVALID hard stop	PASS	mutation acceptance
Base / Upper parity	PASS	snapshots e Steps 8.1/8.2
Persistence / AI update	PASS	registry acceptance
BOM / cut-list / nesting	PASS	snapshots e acceptance downstream
Joinery / Machining	PASS	acceptance downstream
Mutation checks	PASS	logs 09–13
TypeScript new errors	PASS — zero novos	log 14
Vitest	PASS — 599 testes	log 15
Production build	PASS — exit code 0	log 16

12. Critério de aprovação e encerramento

A Stage 9 atende ao critério de que adicionar uma nova ModuleDefinition profissional não exige editar `buildCarcass()` ou `buildDoors()` para inserir novos `if moduleDefinitionId === X`. A nova seleção segue DEFINITION → PROFILE → RULES → PURE RESOLVERS → BUILDER → PART DEFINITIONS.

A missão termina aqui. Stage 10, gaveteiro, torre, nova família, CAM/CNC, G-code e melhorias de render não foram iniciados.